

SEO Audit Agent: AI-Powered Technical SEO Crawler

Project Description:

SEO Audit Agent is an enterprise-grade, autonomous website SEO auditing tool built with Python, Ollama LLM (Mistral), and Google Sheets reporting. The platform performs a full technical SEO crawl across any target domain and delivers structured, prioritised findings through a rich CLI interface, a REST API, and a real-time Google Sheets dashboard.

The system comprises several specialised modules working in concert: a BFS Crawler that asynchronously discovers all reachable internal URLs; a Parallel Checkers Suite that concurrently analyses metadata structures, keyword density, broken links, and readability scores per page; an LLM Reasoner that packages consolidated page errors and generates prioritised markdown remedies via the Mistral model; and a Google Sheets Sync module that compiles run logs for visual trend metrics.

Built with Flask and deployed as a REST API, the platform also ships with a premium Semrush-styled marketing landing page featuring live pipeline simulators, score dials, and dynamic filter dashboards. With secure credential management via .env and Google Cloud Service Accounts, SEO Audit Agent empowers developers, SEO professionals, and businesses to conduct deep website audits with confidence.

Scenarios:

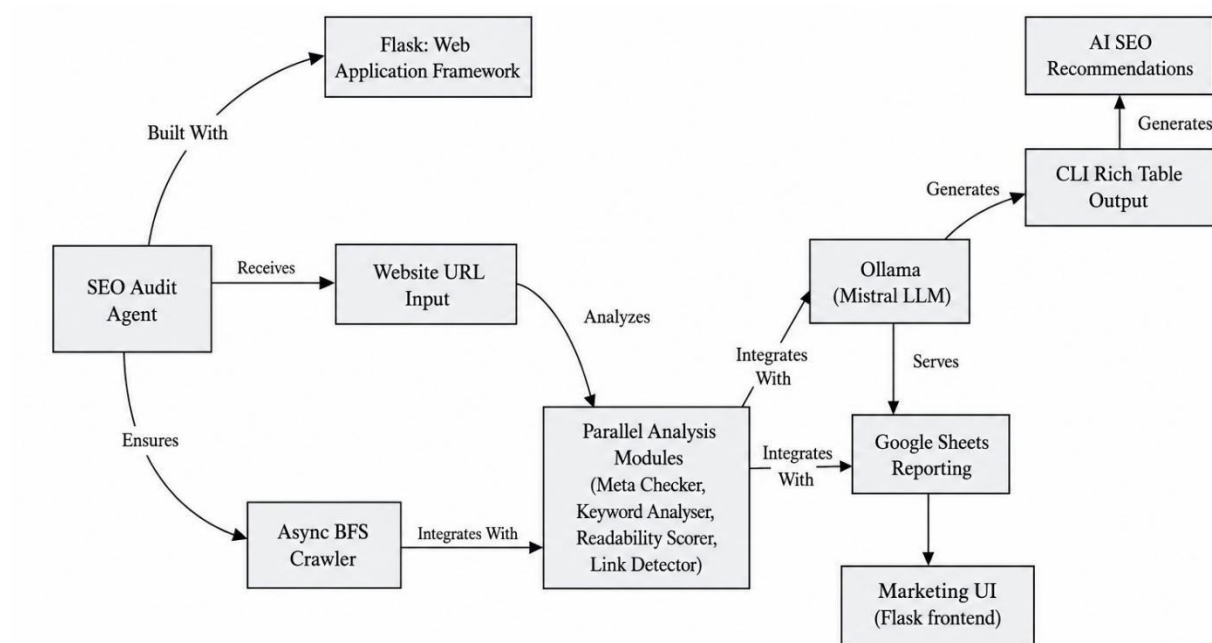
Scenario 1: A developer runs a CLI audit on a small business website. The crawler discovers 45 internal pages, the parallel checkers flag missing Open Graph tags and low readability scores, and the LLM generates CRITICAL and HIGH priority fix recommendations in seconds — all displayed as a formatted Rich table in the terminal.

Scenario 2: An SEO agency integrates the REST API into their client dashboard. They POST a target domain to /audit and receive a structured JSON response with per-page scores, meta issue counts, broken link lists, readability grades, and LLM-generated actionable fixes for each URL.

Scenario 3: A content manager runs an audit with a Google Sheet ID supplied. The agent crawls the site, analyses all pages in parallel, and writes the complete results to a Google Sheets dashboard — creating a living document that tracks SEO health over time across multiple audit runs.

Scenario 4: A user opens the marketing landing page at /marketing/index.html. They experience a cinematic hero terminal with real-time crawling simulations, interactive score dials and radial gauges, a live concurrent pipeline particle flow, and toggle-able mint, dark, and matrix CLI color palettes — all without running the backend.

Technical Architecture:



Module	Responsibility
crawler.py	Async BFS crawler — discovers all internal pages
meta_checker.py	Validates title, meta desc, canonical, Open Graph tags
keyword_analyser.py	Detects overstuffed and under-optimised keywords
readability.py	Flesch and Gunning Fog readability scoring
link_detector.py	Async HEAD requests to detect broken internal/external links
llm_suggestions.py	Generates prioritised fix recommendations via Ollama/Mistral
sheets_writer.py	Writes structured audit data to Google Sheets
api.py	Flask REST API exposing /audit and serving marketing UI
orchestrator.py	Coordinates parallel page analysis with concurrency controls

Pre-requisites:

- **Python 3.10+ and pip for Dependency Management:** <https://docs.python.org/3/>
- **Ollama/Mistral LLM Setup:** <https://ollama.com/docs>
- **Flask Framework Knowledge:** <https://flask.palletsprojects.com/>
- **Google Sheets API Integration:** <https://developers.google.com/sheets/api>
- **Async HTTP and HTML Parsing:** <https://docs.aiohttp.org/> and <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- **Readability Scoring:** <https://pypi.org/project/textstat/>
- **Version Control with Git:** <https://git-scm.com/doc>
- **Development Environment Setup:** <https://flask.palletsprojects.com/en/stable/installation/>
- **Local LLM Deployment:** <https://ollama.com/download>

Project Workflow:

Activity 1: Model Selection and Architecture

- **Activity 1.1:** Research and select the appropriate LLM model (Mistral via Ollama) for SEO recommendation generation.
- **Activity 1.2:** Define the multi-module architecture, detailing async crawling, parallel analysis, and LLM integration.
- **Activity 1.3:** Outline interaction flows between the CLI/API entry points, orchestrator, analysis modules, and output writers.
- **Activity 1.4:** Set up the development environment, installing all Python dependencies from requirements.txt.

Activity 2: Core Module Development

- **Activity 2.1:** Implement the async BFS Crawler to discover all internal pages within a domain.
- **Activity 2.2:** Build the parallel analysis suite: meta checker, keyword analyser, readability scorer, and link detector.

Activity 3: Orchestrator and API Development

- **Activity 3.1:** Write the orchestrator in orchestrator.py to coordinate bounded concurrent page analysis with asyncio Semaphores.

Activity 4: Frontend and Marketing Page Development

- **Activity 4.1:** Design and develop the marketing landing page using HTML, CSS, and JavaScript with a Semrush-styled aesthetic.
- **Activity 4.2:** Implement the cinematic hero terminal, pipeline simulators, score dials, and live filters in app.js.

Activity 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring environment variables and verifying all dependencies.
- **Activity 5.2:** Deploy the Flask API server publicly and configure Google Sheets credentials for live reporting.

Activity 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, the focus is on selecting the appropriate local LLM for SEO recommendation generation and defining the overall application architecture. The Mistral model running via Ollama was chosen for its strong natural language understanding, zero API cost, and ability to return structured JSON recommendations — key requirements for an automated audit pipeline.

Activity 1.1: Install Ollama and Pull the Mistral Model

Before the application can generate LLM-powered SEO suggestions, Ollama must be installed and the Mistral model must be available locally. Follow the steps below.

- **Step 1 — Install Ollama:** Install Ollama on your system:

```
curl -fsSL https://ollama.com/install.sh | sh
```

- **Step 2 — Pull Mistral:** Pull the Mistral model to your local Ollama instance: `ollama pull mistral`

- **Step 3 — Start Ollama Server:** Start the Ollama server in the background: `ollama serve`

- **Step 4 — Verify:** Verify Ollama is running and the model is available:

```
ollama list
```

```
PS C:\Users\baiya\OneDrive\Documents\Projects\seo_agent> ollama list
NAME                ID                SIZE    MODIFIED
mistral:latest      6577803aa9a0     4.4 GB  4 minutes ago
llama3.2:latest     a80c4f17acd5     2.0 GB  4 days ago
phi3:latest         4f222927938      2.2 GB  3 months ago
qwen2.5:latest      845dbda0ea48     4.7 GB  3 months ago
llama3:latest       365c0bd3c000     4.7 GB  3 months ago
nomic-embed-text:latest 0a109f422b47    274 MB  3 months ago
PS C:\Users\baiya\OneDrive\Documents\Projects\seo_agent> cd seo_agent
```

Activity 1.2: Research and Select the LLM

- Understand the project requirements: the LLM must accept a list of per-page SEO issues and return a JSON array of prioritised fix strings tagged [CRITICAL], [HIGH], [MEDIUM], or [LOW].
- Evaluate Ollama-compatible models for local inference speed, instruction-following accuracy, and JSON output reliability.
- Select Mistral for its strong performance on structured output tasks, low resource footprint, and zero API cost — ideal for an autonomous audit pipeline.
- Configure the model in `config/settings.py` with `ollama_model: str = "mistral"` and `ollama_max_tokens: int = 1024`.

Activity 1.3: Define the Architecture

- Draft the architectural diagram illustrating the flow: CLI/API entry → Crawler → Orchestrator → Parallel Checkers → LLM Suggestions → Output (CLI table / JSON / Google Sheets).
- Detail the crawler's BFS strategy: starting from the root URL, extracting internal links, batching async fetches up to max_concurrent (20), and capping at max_pages (200).
- Outline the orchestrator's bounded concurrency model: asyncio.Semaphore(10) ensures at most 10 pages are analysed simultaneously, preventing memory overload on large sites.
- Describe the LLM integration: llm_suggestions.py builds a per-page issue prompt, POSTs to the local Ollama /api/chat endpoint, and parses the raw JSON array response.

Activity 1.4: Set Up the Development Environment

- **Install Python:** Ensure Python 3.10 or higher is installed:

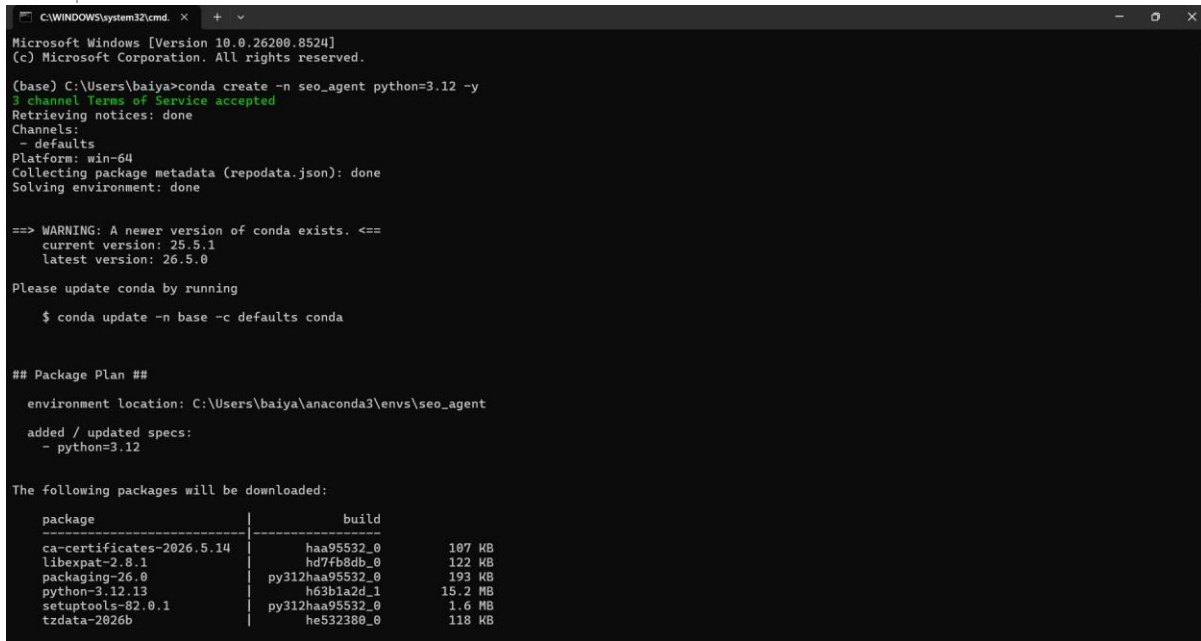
```
python --version
```

- **Create Virtual Environment:** Create and activate a virtual environment:

```
python -m venv seo-env
```

```
source seo-env/bin/activate # macOS/Linux
```

```
seo-env\Scripts\activate # Windows
```



```

C:\WINDOWS\system32\cmd. x +
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

(base) C:\Users\baiya>conda create -n seo_agent python=3.12 -y
3 channel Terms of Service accepted
Retrieving notices: done
Channels:
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 25.5.1
  latest version: 26.5.0

Please update conda by running

  $ conda update -n base -c defaults conda

## Package Plan ##

  environment location: C:\Users\baiya\anaconda3\envs\seo_agent
  added / updated specs:
    - python=3.12

The following packages will be downloaded:

  package                                     build                                size
-----
ca-certificates-2026.5.14                   haa95532_0                          187 KB
libexpat-2.8.1                              hd7f80db_0                          122 KB
packaging-26.0                             py312haa95532_0                     193 KB
python-3.12.13                              h63b1a2d_1                          15.2 MB
setuptools-82.0.1                           py312haa95532_0                     1.6 MB
tzdata-2026b                               he532380_0                          118 KB
  
```

- **Install Dependencies:** Install all dependencies from requirements.txt:

```
pip install -r requirements.txt
```

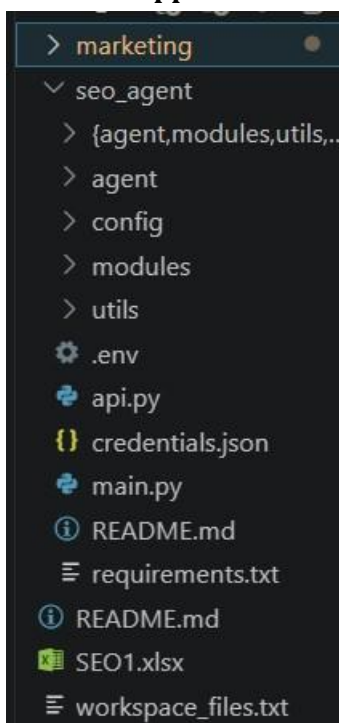
```
PS C:\Users\baiya\OneDrive\Documents\Projects\seo_agent\seo_agent> pip install -r requirements.txt
Requirement already satisfied: flask==3.0.3 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 1)) (3.0.3)
Requirement already satisfied: flask-cors==4.0.1 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 2)) (4.0.1)
Requirement already satisfied: requests==2.32.3 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 3)) (2.32.3)
Requirement already satisfied: beautifulsoup4==4.12.3 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 4)) (4.12.3)
Requirement already satisfied: aiohttp==3.9.5 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 5)) (3.9.5)
Collecting textstat==0.7.3 (from -r requirements.txt (line 6))
  Using cached textstat-0.7.3-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: gspread==6.1.2 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 7)) (6.1.2)
Requirement already satisfied: google-auth==2.30.0 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 8)) (2.30.0)
Requirement already satisfied: lxml==5.2.2 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 9)) (5.2.2)
Requirement already satisfied: tldextract==5.1.2 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 10)) (5.1.2)
Requirement already satisfied: rich==13.7.1 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 11)) (13.7.1)
Requirement already satisfied: python-dotenv==1.0.1 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from -r requirements.txt (line 12)) (1.0.1)
Requirement already satisfied: Werkzeug==3.0.0 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from flask==3.0.3->-r requirements.txt (line 1)) (3.1.8)
Requirement already satisfied: Jinja2==3.1.2 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from flask==3.0.3->-r requirements.txt (line 1)) (3.1.6)
Requirement already satisfied: itsdangerous==2.1.2 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from flask==3.0.3->-r requirements.txt (line 1)) (2.2.0)
Requirement already satisfied: click==8.1.3 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from flask==3.0.3->-r requirements.txt (line 1)) (8.4.1)
Requirement already satisfied: blinker==1.6.2 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from flask==3.0.3->-r requirements.txt (line 1)) (1.9.0)
Requirement already satisfied: charset-normalizer<4,>=2 in C:\Users\baiya\anaconda3\envs\seo_agent\Lib\site-packages (from requests==2.32.3->-r requirements.txt (line 3)) (3.4.7)
```

- **Configure Environment:** Create a .env file in the seo_agent/ directory:

```
GOOGLE_SHEETS_CREDENTIALS=credentials.json
GOOGLE_SHEET_ID=your_google_sheet_id_here
```

```
seo_agent > .env
1  GOOGLE_SHEETS_CREDENTIALS=credentials.json
2  GOOGLE_SHEET_ID=1A5vfDL5fdp1Lvy7cST8L7I4xAXuz20hCeFxi6ijIo
```

- **Application Structure:** Project directory structure:



Milestone 2: Core Module Development

Milestone 2 focuses on building the five core analysis modules that power the SEO audit pipeline. Each module is implemented as a standalone Python file under /modules, designed for concurrent execution via `asyncio.gather()` inside the orchestrator.

Activity 2.1: Implement the Async BFS Crawler

The Crawler class in `crawler.py` performs a breadth-first search across the target domain, asynchronously fetching pages in batches up to `max_concurrent` (20) and respecting a `max_pages` cap (200).

- **URL Normalisation:** The static method `_normalize_root()` strips trailing slashes, corrects common scheme typos (`ttps://`, `ttp://`), and defaults to `https://` when no scheme is provided.
- **Internal Link Detection:** `_is_internal()` uses `tldextract` to compare registered domains, ensuring only same-domain links are followed.
- **Async Batch Fetching:** `crawl()` drains the BFS queue in batches of `max_concurrent`, running `asyncio.gather()` over `aiohttp` HEAD/GET requests with configurable timeouts.
- **HTML Extraction:** Only responses with content-type containing `html` are decoded and stored, saving memory on non-HTML assets.

```
class Crawler:
    def __init__(self, root: str):
        self._root = self._normalize_root(root)
        self._domain = tldextract.extract(self._root).registered_domain
        self._visited: set[str] = set()
        self._queue: deque[str] = deque([self._root])

    @staticmethod
    def _normalize_root(root: str) -> str:
        """
        Normalize a user-supplied target URL:
        - Strip trailing slashes/whitespace.
        - Fix common scheme typos (e.g. "ttps://" -> "https://", "tp://" -> "http://").
        - If no scheme is present at all, default to "https://".
        - Never double-prefix an already-valid http(s) URL.
        """
        root = root.strip().rstrip("/")
        if not root:
            raise ValueError("Target URL cannot be empty")

        parsed = urlparse(root)
        scheme = parsed.scheme.lower()

        if scheme in ("http", "https"):
            return root
```

```
# Common typo'd schemes – missing a leading character
scheme_fixes = {
    "ttps": "https",
    "ttp": "http",
    "tps": "https",
    "ps": "https",
}

if scheme in scheme_fixes:
    fixed_scheme = scheme_fixes[scheme]
    # parsed.path/netloc already hold everything after "scheme://"
    rest = root.split("://", 1)[1] if "://" in root else root
    return f"{fixed_scheme}://{rest}"

# No recognizable scheme at all – treat the whole string as host/path
if not scheme:
    return f"https://{root}"

# Unknown scheme – fall back to https with the netloc/path portion
rest = root.split("://", 1)[1] if "://" in root else root
return f"https://{rest}"

def _is_internal(self, url: str) -> bool:
    return tldextract.extract(url).registered_domain == self._domain

def _clean(self, url: str) -> str:
    # Defensive fix for accidentally double-prefixed URLs, e.g.
    # "https://https://example.com" or "http://tp://example.com"
    for bad, good in (("https://https://", "https://"), ("http://tp://", "http://"),
                     ("https://tp://", "https://"), ("http://https://", "https://")):
        if url.startswith(bad):
            url = good + url[len(bad):]
            break
```

```
async def _fetch(self, session: aiohttp.ClientSession, url: str) -> tuple[int, str]:
    try:
        async with session.get(
            url,
            allow_redirects=True,
            timeout=aiohttp.ClientTimeout(total=settings.crawl_timeout),
        ) as r:
            html = ""
            if r.content_type and "html" in r.content_type:
                html = await r.text(errors="replace")
            return r.status, html
    except Exception:
        return 0, ""

async def crawl(self) -> dict[str, tuple[int, str]]:
    pages: dict[str, tuple[int, str]] = {}
    headers = {"User-Agent": settings.user_agent}
    connector = aiohttp.TCPConnector(limit=settings.max_concurrent)

    async with aiohttp.ClientSession(connector=connector, headers=headers) as session:
        while self._queue and len(self._visited) < settings.max_pages:
            batch = []
            while self._queue and len(batch) < settings.max_concurrent:
                url = self._clean(self._queue.popleft())
                if url not in self._visited:
                    self._visited.add(url)
                    batch.append(url)
```

Activity 2.2: Build the Parallel Analysis Suite

Meta Checker (meta_checker.py)

- Validates the presence and character length of the <title> tag (min 30, max 65 characters).
- Checks for a non-empty meta description and flags duplicate descriptions across pages using a shared seen_descriptions set.
- Verifies canonical link presence and ensures the href begins with http to flag malformed canonicals.
- Detects missing Open Graph tags: og:title, og:description, and og:image.

```
def check_meta(soup: BeautifulSoup, seen_descriptions: set[str]) -> MetaIssues:
    m = MetaIssues()

    title_tag = soup.find("title")
    if not title_tag or not title_tag.get_text(strip=True):
        m.missing_title = True
    else:
        length = len(title_tag.get_text(strip=True))
        m.title_too_short = length < settings.title_min
        m.title_too_long = length > settings.title_max

    desc_tag = soup.find("meta", attrs={"name": "description"})
    if not desc_tag or not desc_tag.get("content", "").strip():
        m.missing_meta_desc = True
    else:
        content = desc_tag["content"].strip()
        if content in seen_descriptions:
            m.duplicate_meta_desc = True
        else:
            seen_descriptions.add(content)

    canonical = soup.find("link", attrs={"rel": "canonical"})
    if not canonical:
        m.missing_canonical = True
    elif not canonical.get("href", "").strip().startswith("http"):
        m.malformed_canonical = True

    m.missing_og_title = not bool(soup.find("meta", attrs={"property": "og:title"}))
    m.missing_og_desc = not bool(soup.find("meta", attrs={"property": "og:description"}))
    m.missing_og_image = not bool(soup.find("meta", attrs={"property": "og:image"}))
```

Keyword Analyser (keyword_analyser.py)

- Tokenises body text into lowercase alpha words of minimum length 3, filtering a custom stopword list.
- Computes per-keyword density against total token count; flags overstuffed keywords at density > 4%.

- Flags pages as underoptimised if the top keyword density falls below 0.5% or no keywords are found.

```
_STOPWORDS = {
    "the", "a", "an", "and", "or", "but", "in", "on", "at", "to", "for", "of",
    "with", "is", "was", "are", "were", "be", "been", "being", "have", "has",
    "had", "do", "does", "did", "will", "would", "could", "should", "may",
    "might", "this", "that", "it", "its", "from", "by", "not", "are", "if",
    "as", "so", "we", "you", "he", "she", "they", "our", "your", "their",
}

def _tokens(text: str) -> list[str]:
    return [
        w.lower() for w in re.findall(r"\b[a-zA-Z]{%d,}\b" % settings.keyword_min_length, text)
        if w.lower() not in _STOPWORDS
    ]

def analyse_keywords(text: str) -> KeywordResult:
    tokens = _tokens(text)
    if not tokens:
        return KeywordResult(underoptimised=True)

    total = len(tokens)
    counts = Counter(tokens)
    top = counts.most_common(settings.keyword_top_n)

    return KeywordResult(
        top_keywords=[(w, round(c / total, 4)) for w, c in top],
        overstuffed=[w for w, c in top if c / total > settings.density_overuse],
        underoptimised=(top[0][1] / total < settings.density_underuse) if top else True,
    )
```

Readability Scorer (readability.py)

- Implements Flesch Reading Ease: $206.835 - 1.015 * (\text{words/sentences}) - 84.6 * (\text{syllables/words})$.
- Implements Gunning Fog Index: $0.4 * ((\text{words/sentences}) + 100 * (\text{complex_words/words}))$.
- Returns a ReadabilityResult with a human-readable grade label (Very easy through Very difficult) and a below_target flag when Flesch score falls below 50.0.

```
def _syllable_count(word):
    word = word.lower().strip(".,;?!")
    if len(word) <= 3:
        return 1
    word = re.sub(r'(?![^aeiou]es|ed|[^aeiou]e)$', '', word)
    word = re.sub(r'^y', '', word)
    return len(re.findall(r'[aeiou]+' , word)) or 1

def _counts(text):
    sentences = len(re.findall(r'[.!?]+' , text)) or 1
    words = re.findall(r'\b\w+\b', text)
    syllables = sum(_syllable_count(w) for w in words)
    return sentences, len(words) or 1, syllables

def flesch_reading_ease(text):
    sentences, words, syllables = _counts(text)
    return 206.835 - 1.015 * (words / sentences) - 84.6 * (syllables / words)

def gunning_fog(text):
    sentences, words, _ = _counts(text)
    word_list = re.findall(r'\b\w+\b', text)
    complex_words = sum(1 for w in word_list if _syllable_count(w) >= 3)
    return 0.4 * ((words / sentences) + 100 * (complex_words / words))

_GRADES = [
    (90, "Very easy"), (80, "Easy"), (70, "Fairly easy"),
    (60, "Standard"), (50, "Fairly difficult"),
    (30, "Difficult"), (0, "Very difficult"),
]

def _label(score: float) -> str:
    for threshold, label in _GRADES:
        if score >= threshold:
            return label
    return "Very difficult"

def score_readability(text: str) -> ReadabilityResult:
    if not text or len(text.split()) < 30:
        return ReadabilityResult(below_target=True, grade_label="Insufficient content")

    flesch = flesch_reading_ease(text)
    fog = gunning_fog(text)

    return ReadabilityResult(
        flesch_score=round(flesch, 2),
        grade_label=_label(flesch),
        gunning_fog=round(fog, 2),
        below_target=flesch < settings.readability_min,
    )
```

Link Detector (link_detector.py)

- Extracts all href and src links from HTML via extract_links() in html_parser.py.
- Sends async aiohttp HEAD requests with a configurable timeout (8 seconds) and max redirect limit (3).
- Returns BrokenLink dataclass instances for any URL returning HTTP 400+ or raising connection exceptions.

```
async def _check(session: aiohttp.ClientSession, url: str, element: str) -> BrokenLink | None:
    try:
        async with session.head(
            url,
            allow_redirects=True,
            timeout=aiohttp.ClientTimeout(total=settings.link_timeout),
            max_redirects=settings.max_redirects,
        ) as r:
            if r.status >= 400:
                return BrokenLink(url=url, element=element, status_code=r.status)
    except aiohttp.TooManyRedirects:
        return BrokenLink(url=url, element=element, error="too_many_redirects")
    except Exception as exc:
        return BrokenLink(url=url, element=element, error=str(exc)[:80])
    return None

async def detect_broken_links(html: str, page_url: str) -> list[BrokenLink]:
    links = extract_links(parse(html), page_url)
    connector = aiohttp.TCPConnector(limit=settings.max_concurrent)
    async with aiohttp.ClientSession(connector=connector, headers={"User-Agent": settings.user_agent}) as session:
        results = await asyncio.gather(*[_check(session, url, el) for url, el in links])
    return [r for r in results if r is not None]
```

Milestone 3: Orchestrator and API Development

Milestone 3 focuses on the orchestrator that coordinates all analysis modules in parallel across crawled pages, and the Flask REST API that exposes the audit pipeline to external consumers.

Activity 3.1: Writing the Orchestrator (orchestrator.py)

Core Orchestration Logic

- The `run_audit()` coroutine receives a dict of `{url: (status_code, html)}` pairs from the crawler.
- A shared `seen_descriptions` set is passed to all meta checker calls to enable cross-page duplicate detection.
- `asyncio.Semaphore(10)` bounds concurrent page analysis to prevent resource exhaustion on large sites.

Per-page analysis flow inside `_analyse_page()`:

```
soup = parse(html)
text = extract_body_text(soup)
title = soup.find("title")
title_str = title.get_text(strip=True) if title else ""

meta, keywords, readability, broken_links = await asyncio.gather(
    asyncio.to_thread(check_meta, soup, seen_descriptions),
    asyncio.to_thread(analyse_keywords, text),
    asyncio.to_thread(score_readability, text),
    detect_broken_links(html, url),
)
```

- `PageResult.compute_score()` calculates a 0–100 SEO health score: starts at 100 and deducts 8 points per meta issue, 5 per overstuffed keyword, 10 for underoptimised content, 10 for low readability, and up to 30 for broken links.
- LLM suggestions are generated after scoring via `generate_suggestions(result)` which calls the local Ollama Mistral endpoint.

Flask REST API (api.py)

- `POST /audit` — accepts JSON body `{"url": "...", "sheet_id": "..."}` , crawls the domain, runs the full audit, and returns a JSON array of per-page results.
- `GET /` — serves the marketing landing page (`index.html`) from the `/marketing` directory.
- `GET /<path:filename>` — serves all static assets (`style.css`, `app.js`) for the marketing UI.
- `GET /health` — returns `{"status": "ok"}` for health checks.

```
# — Auth routes —
@app.route('/api/signup', methods=['POST'])
def signup():
    data = request.get_json()
    email = (data.get('email') or '').strip().lower()
    password = data.get('password') or ''
    if not email or not password:
        return jsonify({'error': 'Email and password are required'}), 400
    if User.query.filter_by(email=email).first():
        return jsonify({'error': 'Email already registered'}), 409
    user = User(email=email, password=generate_password_hash(password))
    db.session.add(user)
    db.session.commit()
    login_user(user)
    return jsonify({'message': 'Account created', 'email': user.email})

@app.route('/api/login', methods=['POST'])
def login():
    data = request.get_json()
    email = (data.get('email') or '').strip().lower()
    password = data.get('password') or ''
    user = User.query.filter_by(email=email).first()
    if not user or not check_password_hash(user.password, password):
        return jsonify({'error': 'Invalid email or password'}), 401
    login_user(user)
    return jsonify({'message': 'Logged in', 'email': user.email})
```

API response structure per page:

```
return jsonify([
    {
        "url": r.url,
        "status_code": r.status_code,
        "title": r.title,
        "score": r.score,
        "meta_issues": r.meta.count(),
        "meta_detail": r.meta.to_str(),
        "broken_links": len(r.broken_links),
        "keyword_issues": r.keywords.to_str(),
        "overstuffed_keywords": r.keywords.overstuffed,
        "underoptimised": r.keywords.underoptimised,
        "flesch_score": round(r.readability.flesch_score, 1),
        "gunning_fog": round(r.readability.gunning_fog, 1),
        "readability": r.readability.grade_label or "n/a",
        "readability_below_target": r.readability.below_target,
        "suggestions": r.suggestions,
```

Milestone 4: Marketing Frontend Development

Milestone 4 focuses on building a premium Semrush-styled marketing landing page for the SEO Audit Agent. The frontend showcases the tool's capabilities through a cinematic hero terminal, interactive dashboards, and live pipeline visualisations — all implemented in vanilla HTML, CSS, and JavaScript with no external framework dependencies.

Activity 4.1: Designing and Developing the User Interface

HTML Structure (index.html)

- A full-page semantic layout with a cinematic hero section featuring an animated typewriter terminal that simulates live crawling output.
- Expandable feature dashboard cards that expand in-place to reveal mini data tables and module detail panels.
- Score trend and radial gauge sections with interactive visibility score dials and dropdown filter lists per page category.
- A custom page theme footer toggle allowing users to switch between mint, dark, and matrix CLI color palettes instantly.

```
marketing > index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7   <!-- SEO Meta Tags -->
8   <title>SEO Audit Agent - Production-Grade AI-Powered SEO Crawler & Analyzer</title>
9   <meta name="description" content="Audit every page, detect every technical issue, and generate prioritized fixes automatically in minutes. Open-s
10  <meta name="keywords" content="SEO Auditor, AI Crawler, Python SEO agent, Ollama LLM, Google Sheets reporting, Broken Link Detector, Meta Tag Che
11  <meta name="robots" content="index, follow">
12
13  <!-- Open Graph -->
14  <meta property="og:title" content="SEO Audit Agent - AI-Powered Website Audits">
15  <meta property="og:description" content="Crawl your entire site, detect SEO issues, and generate real-time sheets reports. Powered by Ollama LLM.
16  <meta property="og:type" content="website">
17
18  <!-- Stylesheets -->
19  <link rel="stylesheet" href="style.css?v=2.0">
20
21  <!-- Favicon -->
22  <link rel="icon" type="image/png" href="data:image/svg+xml,<svg xmlns=%22http://www.w3.org/2000/svg%22 viewBox=%220 0 100 100%22><circle cx=%2250
23
24  <!-- Lucide Icons -->
25  <script src="https://unpkg.com/lucide@latest"></script>
26 </head>
27 <body>
```

CSS Design System (style.css)

- Mint layout design system with custom CSS properties for the default mint theme, with matrix and dark theme overrides.
- Keyframe animations for the concurrent particle pipeline flow paths, score dial sweeps, and scrolling content reveals.
- Responsive grid and flexbox layouts adapting across desktop, tablet, and mobile screen widths.

```
/* --- Google Fonts --- */
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Outfit:wght@500;700;800;900&display=swap');

/* --- Root Variable System --- */
:root {
  /* Fonts */
  --font-heading: 'Outfit', -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
  --font-body: 'Inter', -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
  --font-mono: 'Consolas', 'Courier New', Courier, monospace;

  /* HSL Mint Theme (Default Light) */
  --bg-color: #EBF5F0;
  --bg-card: #FFFFFF;
  --bg-card-alt: #F4FAF7;
  --text-main: #1A1A2E;
  --text-muted: #57576E;
  --border-color: #D3E7DD;
  --shadow-main: 0 12px 40px rgba(29, 60, 45, 0.08);
  --shadow-inset: inset 0 2px 4px rgba(0,0,0,0.02);

  /* Accents */
  --color-purple: #7C5CBF;
  --color-purple-hover: #674AA6;
  --color-purple-light: #F2ECFB;
  --color-teal: #1DB87E;
  --color-teal-hover: #179666;
  --color-teal-light: #E8FAF2;
  --color-dark-bg: #0D0D1A;
}
```

Activity 4.2: JavaScript Interactivity (app.js)

- Simulated CLI streams: async typing simulators with configurable character-per-second rates and cursor blink animations in the hero terminal.
- Dynamic concurrent pipeline: floating particle elements rendered along SVG path lines with randomised timing to simulate parallel crawling activity.
- Live filters: JavaScript-powered search and category filters for the audit results table section, updating the visible rows in real time without page reloads.
- Score dials: SVG-based radial gauge components driven by JavaScript, animating the sweep arc to match the displayed SEO score values.

```
/* =====
AUTH - Login / Signup
===== */
let authMode = "login";

async function checkAuth() {
  try {
    const res = await fetch("/api/me", { credentials: "include" });
    const data = await res.json();
    if (data.email) {
      document.getElementById("nav-user-email").textContent = data.email;
      document.getElementById("nav-user-email").style.display = "inline";
      document.getElementById("nav-login-btn").style.display = "none";
      document.getElementById("nav-logout-btn").style.display = "inline";
    }
  } catch (e) { /* server not running locally - silently ignore */ }
}

function openAuthModal() {
  document.getElementById("auth-modal").style.display = "flex";
  document.getElementById("auth-error").textContent = "";
}

function closeAuthModal() {
  document.getElementById("auth-modal").style.display = "none";
}

function toggleAuthMode() {
  authMode = authMode === "login" ? "signup" : "login";
  document.getElementById("auth-title").textContent = authMode === "login" ? "Sign In" : "Create Account";
  document.getElementById("auth-submit-btn").textContent = authMode === "login" ? "Continue" : "Create Account";
  document.getElementById("auth-toggle").innerHTML = authMode === "login"
    ? 'Don\'t have an account? <span style="color:#7c3aed;">Sign up</span>'
    : 'Already have an account? <span style="color:#7c3aed;">Sign in</span>';
}
```

Milestone 5: Deployment

Milestone 5 covers deploying the SEO Audit Agent first locally to verify correct operation, and then as a publicly accessible Flask server. The deployment spans two modes: a CLI-only audit and a full API server with marketing UI.

Activity 5.1: Preparing the Application for Local Deployment

Set Up Virtual Environment and Install Dependencies

- **Setup:** Create and activate the virtual environment, then install from requirements.txt: python
-m venv env
source env/bin/activate # macOS/Linux
env\Scripts\activate # Windows pip
install -r requirements.txt

Configure Environment Variables

- **Configure .env:** Create a .env file inside the seo_agent/ directory. The app loads this automatically at startup:

```
GOOGLE_SHEETS_CREDENTIALS=credentials.json  
GOOGLE_SHEET_ID=your_google_sheet_id_here
```

Activity 5.2: Local Testing and Verification

Run CLI Audit

- **CLI Command:** Run an audit on any target domain. The agent crawls, analyses, and prints a Rich table summary:

```
python main.py --url https://example.com
```

- **With Sheets Reporting:** To also write results to Google Sheets, supply the --sheet-id flag:
python main.py --url https://example.com --sheet-id YOUR_SHEET_ID

Run Flask API Server

- **Start Server:** Start the API server locally:

```
PS C:\Users\baiya\OneDrive\Documents\Projects\seo_agent> cd seo_agent  
PS C:\Users\baiya\OneDrive\Documents\Projects\seo_agent\seo_agent> python api.py  
* Serving Flask app 'api'  
* Debug mode: off  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI serve  
r instead.  
* Running on http://127.0.0.1:5000  
Press CTRL+C to quit
```

- **Audit via API:** POST an audit request to the API:

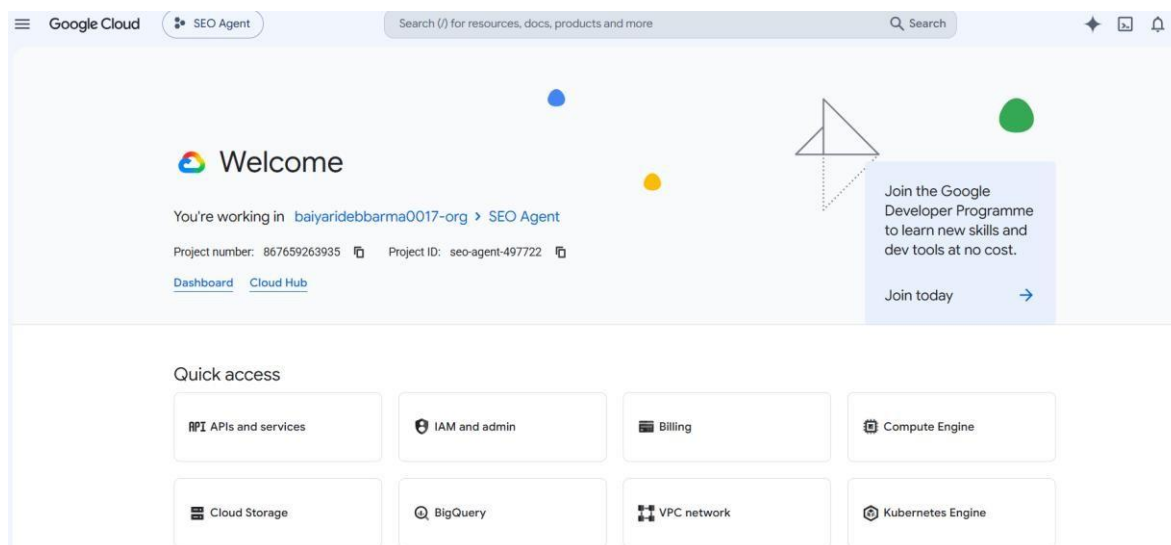
```

127.0.0.1 - - [12/Jun/2026 06:13:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Jun/2026 06:13:00] "GET /style.css?v=2.0 HTTP/1.1" 200 -
127.0.0.1 - - [12/Jun/2026 06:13:00] "GET /app.js?v=2.0 HTTP/1.1" 200 -
127.0.0.1 - - [12/Jun/2026 06:26:29] "GET / HTTP/1.1" 304 -
127.0.0.1 - - [12/Jun/2026 06:26:29] "GET /style.css?v=2.0 HTTP/1.1" 304 -
127.0.0.1 - - [12/Jun/2026 06:26:29] "GET /app.js?v=2.0 HTTP/1.1" 200 -
127.0.0.1 - - [12/Jun/2026 06:27:05] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [12/Jun/2026 06:27:05] "OPTIONS /audit HTTP/1.1" 200 -
Analysing 1 pages...
  
```

Interact with the full CLI and API modes — test CLI table output, JSON API responses, marketing UI at <http://localhost:5000>, and Google Sheets writing — to verify the complete audit pipeline is functioning correctly.

Activity 5.3: Google Sheets Integration Setup

1. Go to Google Cloud Console <https://console.cloud.google.com>



2. Create or select a project

- Click the project dropdown at the top
- Click "New Project" → give it a name → "Create"



3. Enable the Google Sheets API

- Go to **APIs & Services** → **Library**
- Search "Google Sheets API" → Click it → "Enable"
- Also enable **Google Drive API** the same way

Product details



Google Drive API

[Google Enterprise API](#)

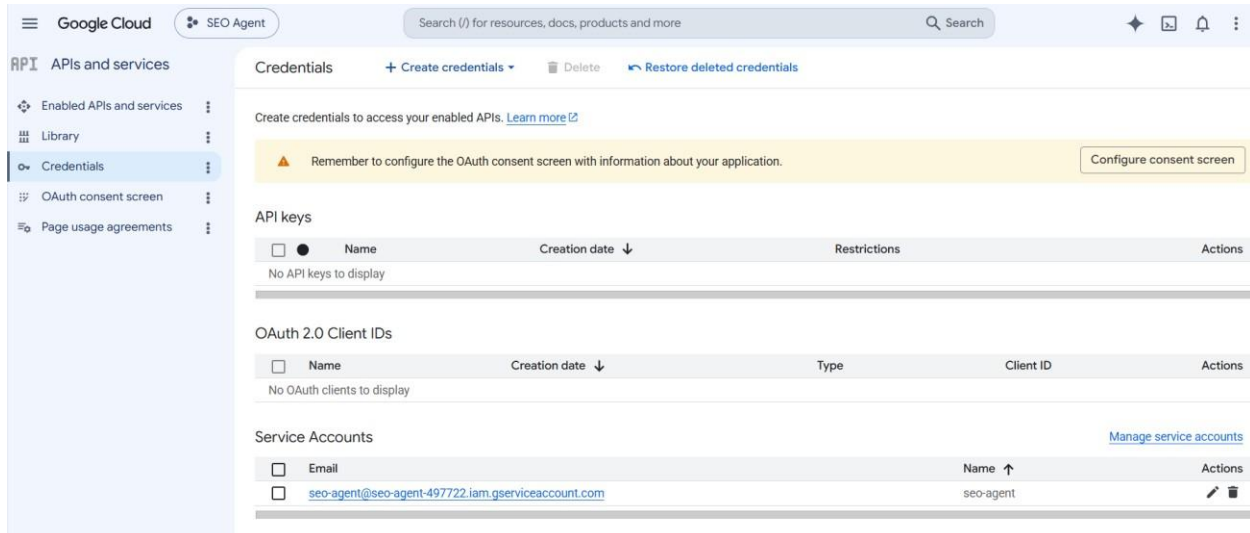
Create and manage resources in Google Drive.

Enable

[Try this API](#)

4. Create the Service Account

- Go to **APIs & Services** → **Credentials**
- Click "Create Credentials" → "Service Account"
- Give it a name (e.g. seo-audit-agent) → "Create and Continue"
- Skip the optional role/user steps → "Done"



Google Cloud | SEO Agent | Search (/) for resources, docs, products and more

APIs and services

- Enabled APIs and services
- Library
- Credentials**
- OAuth consent screen
- Page usage agreements

Credentials | + Create credentials | Delete | Restore deleted credentials

Create credentials to access your enabled APIs. [Learn more](#)

Remember to configure the OAuth consent screen with information about your application. [Configure consent screen](#)

API keys

☐	Name	Creation date ↓	Restrictions	Actions
No API keys to display				

OAuth 2.0 Client IDs

☐	Name	Creation date ↓	Type	Client ID	Actions
No OAuth clients to display					

Service Accounts [Manage service accounts](#)

☐	Email	Name ↑	Actions
☐	seo-agent@seo-agent-497722.iam.gserviceaccount.com	seo-agent	Edit Delete

5. Download the JSON key

- Click your newly created service account
 - Go to the **Keys** tab
 - "Add Key" → "Create new key" → **JSON** → "Create"
 - A credentials.json file downloads automatically
- 6. Place it in your project** `bash mv ~/Downloads/your-key-file.json seo_agent/credentials.json`

```
seo_agent > .env
1  GOOGLE_SHEETS_CREDENTIALS=credentials.json
2  GOOGLE_SHEET_ID=1A5vfDL5fdpkp1Lvy7cST8L7I4xAxuz20hCeFxi6ijIo
```

7. Share your Google Sheet with the service account

- Open your Google Sheet
- Click "Share"
- Paste the service account email (looks like seo-audit-agent@yourproject.iam.gserviceaccount.com)
- Give it **Editor** access → "Send"



General access

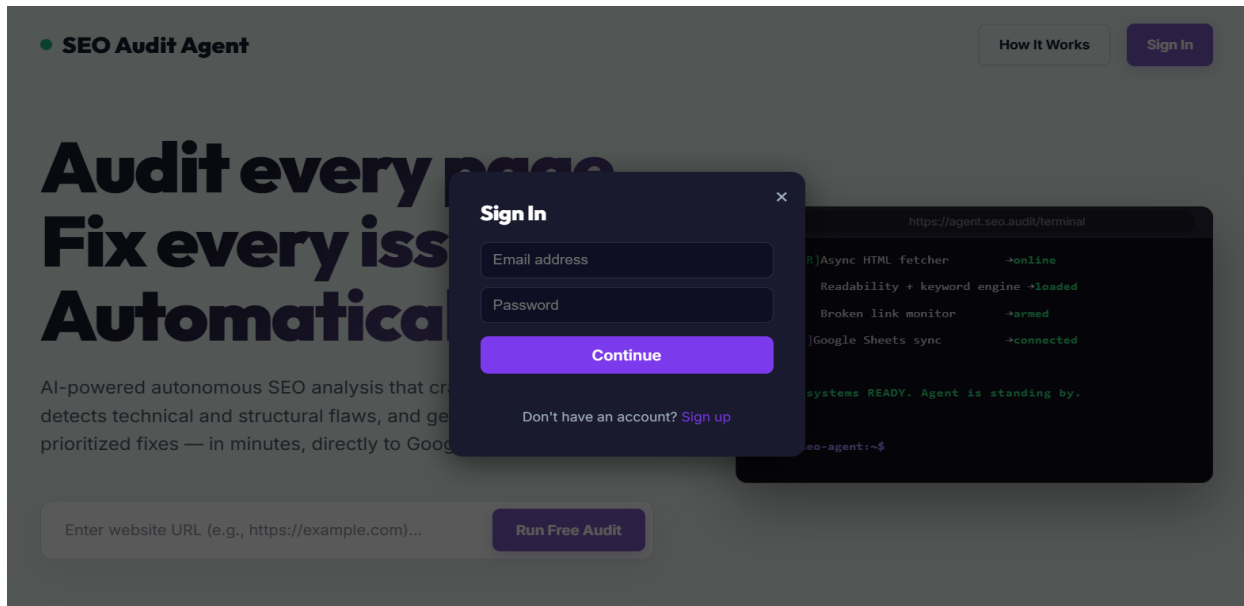


Important Notes:

- Ollama must be running (ollama serve) before starting any audit. If it is not running, LLM suggestions will fall back to [HIGH] Could not generate LLM suggestions.
- The Mistral model must be pulled before first use: ollama pull mistral.
- The Google Sheets write step is entirely optional. Without --sheet-id, the audit runs fully and outputs to the terminal only.
- On large sites approaching the 200-page crawl cap, audit runtime scales roughly linearly with the number of pages and available system resources.

Exploring the Website's Web Pages: Home/Generator Page:

Description: The marketing landing page serves as both the public face of the SEO Audit Agent and a live demonstration of its capabilities. Built with a Semrush-inspired aesthetic, it features a dark mint design system with animated particle pipelines, cinematic hero terminal sequences, expandable feature dashboards, and interactive score gauges — all designed to communicate the power of the underlying audit engine visually and immediately.



Terminal Section:

Description: A full-viewport hero section containing a typewriter-animated CLI terminal that simulates a live crawl in progress — displaying URL discovery messages, module progress indicators, and a final audit score reveal. The terminal is styled with monospace fonts, a dark background, and mint accent colors to evoke a real command-line environment.

● **SEO Audit Agent**

[How It Works](#)
[Sign In](#)

Audit every page. Fix every issue. Automatically.

AI-powered autonomous SEO analysis that crawls your entire website, detects technical and structural flaws, and generates production-ready prioritized fixes — in minutes, directly to Google Sheets.

```
https://agent.seo.audit/terminal

[CRAWLER] Async HTML fetcher      →online
[NLP]   Readability + keyword engine →loaded
[LINKS] Broken link monitor        →armed
[REPORT] Google Sheets sync        →connected

▶ All systems READY. Agent is standing by.

admin@seo-agent:~$
```

[Run Free Audit](#)

Feature Dashboard Section:

Description: A grid of expandable feature cards, each representing one analysis module. Clicking a card expands it in-place to reveal a mini data table showing example outputs — flagged meta issues, keyword density readings, readability grades, and broken link counts. The concurrent pipeline particle flow (floating dot elements along SVG paths) is rendered behind the cards to visually convey parallel execution.

ENTERPRISE AUDITING SUITE

GET AUDITED. GET FIXED. BE THE TOP RESULT.

CRAWLING MODULE 1

+

Meta Tags Analyzer

Inspect every page for missing titles, duplicate descriptions, broken canonicals, and absent Open Graph fields automatically.

CRAWLING MODULE 2

+

Keyword Analyzer

Analyse keyword frequency across every page. Flag overstuffing pages and thin content automatically.

CRAWLING MODULE 3

+

Readability Scorer

Score every page against Flesch-Kincaid and Gunning Fog. Flag content too complex for your audience.

CRAWLING MODULE 4

+

Broken Link Detector

CRAWLING MODULE 5

+

Ollama LLM Engine

CRAWLING MODULE 6

+

Google Sheets

PARALLEL PROCESSING ENGINE

Bigger insight. Faster results.

The autonomous agent crawls all reachable internal pages concurrently using async BFS algorithms, executes 5 analysis checkers in parallel, and generates a cohesive report dashboard in minutes — not hours.

200
pages

Concurrent
Crawl capacity

5
checks

Checks
Executed Parallel

< 1.8s
response

Average LLM
Reasoning Speed

PAGES
CRAWLED

147

ISSUES
FOUND

312

LLM CALLS
MADE

147

AVG SEO
SCORE

74/100



CRAWL



META



KEYWORDS



READABILITY



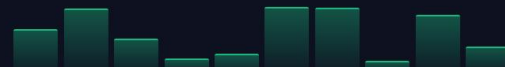
LINKS



LLM



SHEETS



Audit report in Google Sheets:

SEO1

Saved to Drive

File

Edit

View

Insert

Format

Data

Tools

Extensions

Help

100%

123

Default...

10

B

Z

<

SEO1

File Edit View Insert Format Data Tools Extensions Help

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

123 Arial

10

B I U A

100%

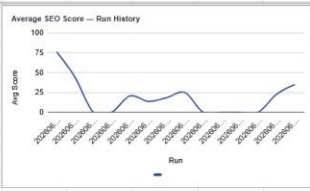
123 Arial

10

B I U A

100%

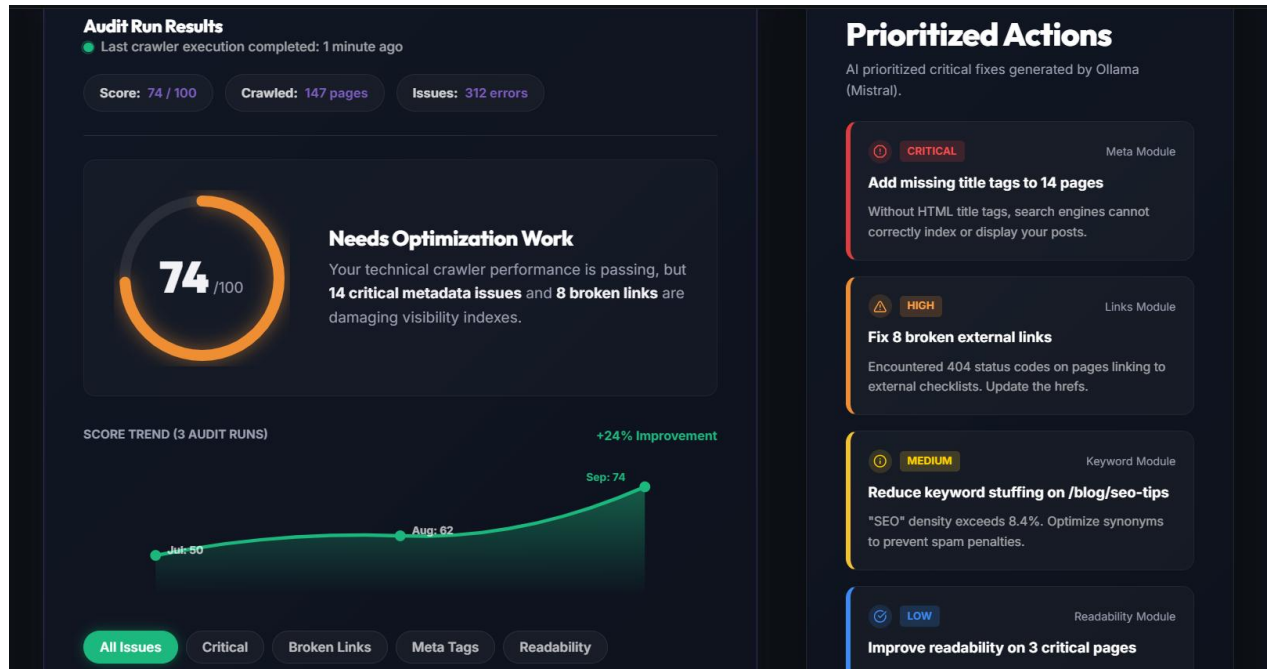
123 Arial</

SEO1													
File Edit View Insert Format Data Tools Extensions Help													
Q Menus 100% 123 Arial 10 B I U A													
1	Run ID	Timestamp (UTC)	Target Domain	Pages Audited	Avg SEO Score	Critical Pages (<50)	Total Meta Issues	Total Broken Links	Status				
2	20260612-44ZT	2026-06-12 02:03 UTC	webscraper.io	1	76	0	0	2	✓ Complete				
3	20260613-5NW7	2026-06-13 07:49 UTC	books.toscrape.com	201	44.2	137	1034	335	✓ Complete				
4	20260613-2FMS	2026-06-13 07:52 UTC	example.com	1	0	1	6	1	✓ Complete				
5	20260613-LC32	2026-06-13 08:08 UTC	example.com	1	0	1	6	1	✓ Complete				
6	20260613-M10H	2026-06-13 08:41 UTC	quotes.toscrape.com	21	20.8	20	126	75	✓ Complete				
7	20260613-350C	2026-06-13 09:23 UTC	info.cem.ch	5	13.8	5	29	314	✓ Complete				
8	20260613-R641	2026-06-13 09:30 UTC	home.web.cem.ch	5	18.4	5	29	271	✓ Complete				
9	20260613-UJCM	2026-06-13 09:35 UTC	info.cem.ch	5	25	5	29	76	✓ Complete				
10	20260613-4T17	2026-06-13 09:46 UTC	example.com	1	0	1	6	1	✓ Complete				
11	20260613-17R0	2026-06-13 09:50 UTC	example.com	1	0	1	6	1	✓ Complete				
12	20260613-YB20	2026-06-13 09:53 UTC	example.com	1	0	1	6	1	✓ Complete				
13	20260613-87ET	2026-06-13 09:55 UTC	example.com	1	0	1	6	1	✓ Complete				
14	20260613-A7V7	2026-06-13 10:09 UTC	httpbin.org	4	22.8	4	24	0	✓ Complete				
15	20260613-3UM9	2026-06-13 10:35 UTC	httpbin.org	1	35	1	5	0	✓ Complete				

	A	B	C	D	E
1	 Issues Summary — Ranked by Frequency				
2	Based on audit of 1 page(s). Issues sorted by most affected pages.				
3					
4	Issue Type	Affected Pages	% of Audited Page	Severity	Recommended Action
5	Missing Meta Description	1	100%	CRITICAL	Write a 150–160 char meta description per page
6	Missing Canonical Tag	1	100%	HIGH	Add <link rel='canonical' href='...'> to every page
7	Missing og:title	1	100%	MEDIUM	Add Open Graph title for social sharing
8	Missing og:description	1	100%	MEDIUM	Add Open Graph description for social sharing
9	Missing og:image	1	100%	MEDIUM	Add a 1200×630px og:image for link previews
10	Keyword Overstuffing (>4% density)	1	100%	HIGH	Reduce keyword frequency; aim for 1–2% natural density
11	Low Readability (Flesch <50)	1	100%	MEDIUM	Simplify sentences; target 8th-grade reading level
12					
13					
14					
15					

Score and Results Section:

Description: An interactive results panel featuring SVG radial score dials that animate to match SEO health scores, a dropdown filter for selecting page categories, and a sortable results table. Users can toggle between mint, dark, and matrix CLI color themes via the footer controls, with all visual elements adapting instantly via CSS custom properties.



Conclusion:

SEO Audit Agent is an enterprise-grade autonomous auditing platform built with Python, Ollama (Mistral), Flask, and Google Sheets integration. The system combines a high-performance async BFS crawler with a parallel analysis suite covering metadata validation, keyword density analysis, readability scoring, and broken link detection — all coordinated by an asyncio orchestrator with bounded concurrency. LLM-powered suggestions from the local Mistral model deliver prioritised, actionable fix recommendations per page without relying on any external paid API. The project — spanning model selection, module development, orchestration, and deployment — demonstrates how a structured async architecture and targeted local AI can deliver production-grade SEO intelligence for developers and agencies, with clear pathways for future expansion into scheduled audits, multi-site dashboards, and CI/CD pipeline integration.